

Подключение к серверу и отправка данных на него

В прошлом уроке мы смогли подключиться к сети Wi-Fi. Теперь займемся тем, что будем подключаться к серверу и передавать ему данные для дальнейшей обработки. В будущем это нам даст возможность отправки изображения с камеры на образовательный портал, а также получать команды управления машинкой.

Открываем скетч, который мы использовали на прошлом занятии. Первое, что мы сделаем, это создадим переменную, которая будет хранить адрес нашего сервера (добавим в методичку актуальный адрес, этот для примера):

```
const char* serverIP = "http://192.168.1.100:8080"; // Адрес вашего сервера
```

Теперь нам не придется писать этот адрес каждый раз, когда он нам понадобится.

По аналогии сделайте переменные для пароля и имени сети Wi-Fi, к которой мы подключались ранее.

Для связи с сервером нужно стабильное подключение к Wi-Fi. Нужно сделать так, чтобы при любых неполадках плата нам об этом сигнализировала. Для этого нам понадобится функция *loop()*. Что это за функция? С английского это слово означает “петля”, т.е. что-то заикленное. Данная функция после своего окончания вызывается повторно каждый раз до тех пор, пока не отключится питание от платы.

Примечание:

Если брать классический Си/C++, то это выглядело бы так:

```
int main()
{
    setup();

    while(true) {
        loop();
    }
}
```

```
}
```

В функции *loop()* мы будем проверять подключена ли наша плата к Wi-Fi, или же соединение потеряно. Если соединение потеряно, то пытаемся подключиться заново. Все необходимое для этого мы уже использовали в прошлом уроке - попробуйте сначала подумать, как это сделать. Если не знаете как, то ответ будет дальше.

Ответ:

```
if (WiFi.status() == WL_CONNECTED) {  
    Serial.println("Все еще подключены к Wi-Fi");  
} else {  
    // Если соединение потеряно - пытаемся переподключиться  
    Serial.println("Потеряно соединение с Wi-Fi");  
    connectToWiFi();  
}
```

Дополнительно:

Сделайте функцию *blinkLED(int count, int delayTime)*, которая позволит по переданным параметрам изменять задержку между включением/выключением светодиода, а также задавать количество миганий. Понадобится цикл *for*, а также код ожидания подключения к *Wi-Fi* из прошлого урока.

Загружаем скетч. Мы должны увидеть следующие сообщения в консоли:

```
22:37:39.041 -> Wi-Fi подключен!  
22:37:39.041 -> IP адрес: 192.168.31.138  
22:37:39.041 -> Все еще подключены к Wi-Fi  
22:37:39.844 -> Все еще подключены к Wi-Fi  
22:37:40.647 -> Все еще подключены к Wi-Fi  
22:37:41.449 -> Все еще подключены к Wi-Fi  
22:37:42.250 -> Все еще подключены к Wi-Fi  
22:37:43.052 -> Все еще подключены к Wi-Fi  
22:37:43.856 -> Все еще подключены к Wi-Fi  
22:37:44.626 -> Все еще подключены к Wi-Fi
```

Теперь настало время сформировать запрос к серверу и отправить ему данные. Создадим функцию `sendDataToServer()`, в котором у нас будет код соединения с сервером.

Для работы с запросами будем использовать объект класса **HTTPClient** — класс из библиотеки `arduino-esp32`, который позволяет устройствам ESP32 делать запросы к веб-серверам по протоколу HTTP/HTTPS:

```
HTTPClient http; // Создаем "почтальона" для отправки данных
```

Указываем по какому адресу (URL) должна подключиться плата к серверу. Для этого нам нужен IP сервера, который мы задали в переменной `serverIP` в самом начале, а также номер порта (порт используем 8080):

```
String url = "http://" + String(serverIP) + ":8080";
```

Теперь можем начинать соединение с помощью метода `begin()`:

```
http.begin(url);
```

Формируем заголовки запроса:

```
http.addHeader("Content-Type", "text/plain");  
http.addHeader("Connection", "close");
```

Подготовим данные для отправки. Отправим на сервер, например, данные о нашей плате:

```
String postData = "";  
postData += "===== ДАННЫЕ ESP32-CAM =====\n";  
postData += "Устройство: ESP32-CAM\n";  
postData += "MAC адрес: " + WiFi.macAddress() + "\n";  
postData += "IP адрес: " + WiFi.localIP().toString() + "\n";  
postData += "Время работы: " + String(millis() / 1000) + " сек\n";  
postData += "===== \n";
```

Просто отправляем текст, что мы и указали в Header нашего запроса - text/plain. Вместо обычного текста мы могли отправить JSON-запрос, но для демонстрации хватит и текста. Скопируйте данный код, чтобы не тратить время на его написание.

“close” мы используем для того, чтобы сервер не держал соединение открытым ожидая новых данных от ESP32.

Дополнительно: отправьте данный запрос в консоль.

Немного о том, как мы можем взаимодействовать с сервером по http протоколу. На данный момент для нас важны два вида запросов: GET и POST.

Если кратко: GET используем для запроса данных от сервера, POST - для отправки. Т.к. мы хотим отправить данные на сервер, то используем метод POST:

```
int httpCode = http.POST(postData);
```

httpCode - это ответ переменная для хранения ответа от сервера об успешности операции. Примеры ответов:

- 1) Код 200 = всё ок;
- 2) Код 404 = страница не найдена;
- 3) Код 500 = ошибка сервера.

Создадим простую проверку. Если *httpCode* > 0, то сервер доступен. Если сервер отключен, то в ответе придет -1. Предлагаю это сделать самостоятельно.

На наш POST-запрос сервер пошлет некий ответ в виде текста. Попробуйте его прочитать после проверки условия *httpCode* > 0 с помощью *http.getString()*.

Задание: *http.getString()* возвращает значение типа String. Ранее в коде мы создавали такую переменную и выводили ее содержимое в консоль. Прочитайте ответ сервера и выведите его в консоль.

Ответ:

```
if(httpCode > 0) {  
    Serial.print("Данные получены! Код: ");  
    Serial.println(httpCode);  
  
    String response = http.getString();  
  
    Serial.println("\nДААННЫЕ С СЕРВЕРА:");  
    Serial.println("=====");  
  
    // Выводим красиво  
    Serial.println(response);  
}
```

Задание: подумайте, какие логи вам могут понадобиться для отладки кода и добавьте их в код.

После того, как мы получили ответ от сервера нужно закрыть HTTP-соединение с помощью метода *end()*:

```
http.end();
```

На этом мы завершили функцию *sendDataToServer()*. Добавьте ее вызов в основной цикл программы после проверки соединения по Wi-Fi.

Загрузите код на плату и посмотрите в консоль. У вас должно быть примерно такое:

```
00:23:23.424 -> Инициализация
00:23:23.424 ->
00:23:23.424 -> Подключаемся к Wi-Fi сети: Подключаемся
00:23:24.002 -> ....
00:23:25.512 -> Wi-Fi подключен!
00:23:25.512 -> IP адрес: 192.168.31.138
00:23:25.512 -> Все еще подключены к Wi-Fi
00:23:25.512 -> ОТПРАВКА ДАННЫХ НА СЕРВЕР
00:23:25.512 -> Отправляемые данные:
00:23:25.545 -> ===== ДАННЫЕ ESP32-CAM =====
00:23:25.545 -> Устройство: ESP32-CAM
00:23:25.545 -> MAC адрес: FC:B4:67:4E:8E:04
00:23:25.545 -> IP адрес: 192.168.31.138
00:23:25.545 -> Время работы: 2 сек
00:23:25.545 -> =====
00:23:25.545 ->
00:23:25.545 -> Отправка... Успешно! Код: 200
00:23:25.736 -> Ответ сервера: Ты справился!
00:23:25.736 -> Время: 00:23:25
00:23:25.736 -> Статус: OK
```

У вас должен быть выведен ответ сервера и данные, которые мы готовили к отправке. Остальные логи у нас могут не совпадать.

Данные мы отправили на сервер, а как насчет получить их от него с помощью запроса GET? Нам для этого понадобится некоторая часть кода от функции `sendDataToServer()`, что радует! Создадим функцию `requestDataFromServer()`. Добавим в нее следующий код:

```
HTTPClient http; // Создаем "почтальона" для отправки данных
String url = "http://" + String(serverIP) + ":8080/getdata";
```

```
http.begin(url);
int httpCode = http.GET\(\);
```

По сравнению с кодом `sendDataToServer()` у нас изменилось следующее - адрес запроса и метод объекта `http`.

Задание: завершить функцию `requestDataFromServer()`. Добавить логирование и с помощью кода *httpCode* понять, пришел ли нам ответ. Если он пришел, вывести ответ в консоль (по аналогии с `sendDataToServer()`).

Функция готова. Добавим ее в основной цикл после `sendDataToServer()`. Загружаем прошивку и смотрим результат:

```
00:43:06.571 -> Wi-Fi подключен!
00:43:06.571 -> IP адрес: 192.168.31.138
00:43:06.571 -> Все еще подключены к Wi-Fi
00:43:06.571 -> ОТПРАВКА ДАННЫХ НА СЕРВЕР
00:43:06.603 -> Отправляемые данные:
00:43:06.603 -> ===== ДАННЫЕ ESP32-CAM =====
00:43:06.603 -> Устройство: ESP32-CAM
00:43:06.603 -> MAC адрес: FC:B4:67:4E:8E:04
00:43:06.603 -> IP адрес: 192.168.31.138
00:43:06.603 -> Время работы: 2 сек
00:43:06.603 -> =====
00:43:06.603 ->
00:43:06.603 -> Отправка... Успешно! Код: 200
00:43:06.828 -> Ответ сервера: Ты справился!
00:43:06.828 -> Время: 00:43:06
00:43:06.828 -> Статус: OK
00:43:06.828 ->
00:43:06.828 -> ЗАПРАШИВАЮ ДАННЫЕ С СЕРВЕРА...
00:43:06.828 -> Данные получены! Код: 200
00:43:06.828 ->
00:43:06.828 -> ДАННЫЕ С СЕРВЕРА:
00:43:06.861 -> =====
00:43:06.861 -> СЕРВЕРНЫЕ ДАННЫЕ:
00:43:06.861 -> Время сервера: 00:43:06
00:43:06.861 -> Дата: 02.02.2026
00:43:06.861 -> Температура CPU: 47°C
00:43:06.861 -> Загрузка памяти: 33%
00:43:06.861 -> Состояние: нормальное
00:43:06.861 -> Последнее обновление: 00:43
```

На данном занятии мы научились отправлять GET и POST запросы на сервер. В следующий раз поработаем с камерой, идущей в комплекте с ESP32-CAM